

KI-Agent-Workflow — destilliert aus Peter Steinbergers Build-Talk, Boris Chernys eigenem Workflow und dem Loops-Konzept

Quellen: [video-summary-TP73qyFWDcY.md](#) ("1,3 Mio. \$ in 30 Tagen: So sieht KI Coding ohne Limits aus"), [video-summary-KWrsLqnB6vA.md](#) ("How Claude Code's Creator Starts EVERY Project"), [video-summary-HASGvvp1M3E.md](#) ("LOOPS statt PROMPTS?")

Zugeschnitten auf Solo-Entwicklung mit Claude Code — nur die Prozesse, die sich direkt umsetzen lassen. Die Punkte 1-5 stammen aus Steinbergers Talk, 6-8 ergänzen Boris Chernys (Claude-Code-Erfinder) eigenen Workflow und das "Loops statt Prompts"-Konzept — beide decken sich stark mit Punkt 1 (Closing the Loop) und 3 (vision.md), weshalb sie hier ergänzt statt als eigene Datei angelegt wurden. Siehe auch [karpathy-claude-md-guidelines.md](#) für eine weitere, unabhängig entstandene Umsetzung derselben Grundidee als copy-paste-fähige CLAUDE.md.

1. Closing the Loop — Agent erst melden lassen, wenn er selbst verifiziert hat

Nicht: Prompt geben → Ergebnis bekommen → selbst testen → Bug melden → wiederholen.
Stattdessen: dem Agenten die Mittel geben, sich selbst zu verifizieren, bevor er sich meldet.

Umsetzung:

- Vor "fertig" melden lassen: Dev-Server starten, Browser öffnen, Feature manuell durchklicken (passt zum `/run-` und `/verify-Skill`, die hier schon verfügbar sind)
- Bei UI-Änderungen: Screenshot/Browstertest einfordern statt "sollte funktionieren" zu akzeptieren
- Permissions so setzen, dass der Agent Server starten, Tests laufen lassen und Debugging-Tools nutzen darf, ohne jedes Mal nachzufragen

2. Autoreview-Loop — Review-Zyklus an den Agenten abgeben

Nicht: einmal reviewen lassen, Bugs fixen, fertig.
Stattdessen: der Agent reviewt sein eigenes Ergebnis so lange, bis er nichts mehr findet.

Umsetzung:

- Nach größeren Changes `/code-review` (oder bei wichtigeren Branches `/code-review high/ultra`) nicht nur einmal, sondern als Schleife verstehen: Findings fixen lassen → erneut reviewen lassen → erst dann als fertig betrachten
- Diese Schleife explizit einfordern statt nach dem ersten Fix zufrieden zu sein

3. vision.md neben CLAUDE.md führen

`agents.md/CLAUDE.md` beschreibt *wie* gearbeitet wird. `vision.md` beschreibt *wohin* das Projekt soll — hilft dem Agenten bei Entscheidungen, die nicht eindeutig aus dem Code ableitbar sind (z. B.

"welche Features passen rein, welche nicht").

Umsetzung:

- Bei Projekten, die über ein Wochenend-Experiment hinausgehen, ein kurzes `vision.md` anlegen: Projektziel, Richtung, offene Diskussionspunkte
- Nicht für Wegwerf-Skripte oder einmalige Spielereien — lohnt sich erst, wenn das Projekt über mehrere Sessions wächst

4. Ideen aus eigener Reibung statt aus der Luft

Peters Tools (ClawSweeper, Crabbox, GOG CLI) entstanden alle aus echtem, wiederkehrendem Ärger — nicht aus Spekulation.

Umsetzung:

- Wenn ein manueller Schritt mehr als 1-2x nervt → das ist der Kandidat für ein eigenes Skill/Script, nicht eine hypothetische Idee
- Konkret im Claude-Code-Kontext: wiederkehrende manuelle Abläufe als Skill unter `~/.claude/skills/` festhalten (wie es mit `watch` schon existiert)

5. Nicht alles muss systemkritisch sauber sein

Vieles an internen Tools/Skripten ist nicht geschäftskritisch → bewusst "vibe-coden" erlauben, ohne jede Zeile zu prüfen.

Umsetzung:

- Bei persönlichen Tools/Automatisierungen (nicht bei production-facing Code) Review-Tiefe bewusst reduzieren — Zeit sparen für die Dinge, die wirklich Fehler kosten dürfen

6. Plan Mode als Standardeinstieg (Boris Cherny)

Boris startet ca. 80% seiner Sessions im Plan Mode, statt sofort bauen zu lassen — "move slow to move fast".

Umsetzung:

- Vor größeren Aufgaben Plan Mode nutzen (zweimal Shift+Tab) statt direkt drauflos zu prompten
- Prompt-Vorlage: *"Before we start building, interview me about this. What are the core problems this solves? Who is this for? What does success look like? And what should this not do? Summarize it back to me before we write any code."*

7. CLAUDE.md schlank halten statt endlos anreichern

Boris' eigene CLAUDE.md ist laut ihm nur wenige tausend Tokens lang. Statt immer mehr Regeln anzuhäufen, bei Bloat eher zurückschneiden — Modelle werden von Version zu Version besser, viele alte Regeln werden überflüssig und zu viele Anweisungen verwirren eher als sie helfen.

Umsetzung:

- Gelegentlich prüfen/aufräumen lassen: *"Update my CLAUDE.md to remove anything that's no longer needed, contradictory, duplicate information, or unnecessary bloat impacting effectiveness."*

8. Von Prompts zu Loops/Goals

Statt eines Hin-und-Her (Prompt → warten → nachbessern) ein Goal formulieren, das der Agent autonom bis zur Erfüllung verfolgt (Claude Code: `Schedule`; ähnliches Konzept bei Codex: Automations).

Umsetzung:

- Goals kurz, konkret und messbar formulieren (Gegenbeispiel aus dem Video: "Baue mir meine Website neu" lief 20 Stunden autonom, Ergebnis passte am Ende trotzdem nicht — große Ziele in überprüfbare Teilziele zerlegen)
- Gut geeignet für klar abgrenzbare Listenaufgaben (offene PRs/Bugs abarbeiten, Inbox aufräumen), nicht für vage Wünsche
- Judgment/Taste bleiben menschlich: entweder klare Erfolgskriterien vorgeben oder die Bewertung explizit einem zweiten LLM als "Judge" übergeben

Bewusst nicht übernommen

- **ClawSweeper/Crabbox als Cloud-Infrastruktur** — macht nur bei Projekten mit hohem Issue-/Test-Volumen Sinn (35k+ Issues, große Testsuiten), nicht für Solo-Projekte in diesem Maßstab
- **Token-Maxing-Mentalität** — laut Video wird Token-Sparsamkeit zunehmend wichtiger; Ankündigung eines Folgevideos zu Prompt Caching — bei Gelegenheit gegenchecken, ob das neue Erkenntnisse für den eigenen Workflow bringt
- **Radikales Löschen der ganzen CLAUDE.md bei Bloat (Boris' eigene Praxis)** — der vorsichtiger Mittelweg (gezielt aufräumen lassen statt komplett neu anfangen) passt besser zu bestehenden, bereits funktionierenden Projekt-Setups

Quelle: ai-agent-workflow.md